Solution: Largest Unique Number (easy)

Problem Statement

Given an array of integers, identify the highest value that appears only once in the array. If no such number exists, return -1.

Examples:

1. **Example 1**:
    - Input: [5, 7, 3, 7, 5, 8]
    - Expected Output: 8
    - Justification: The number 8 is the highest value that appears only once in the array.
2. **Example 2**:
    - Input: [1, 2, 3, 2, 1, 4, 4]
    - Expected Output: 3
    - Justification: The number 3 is the highest value that appears only once in the array.
3. **Example 3**:
    - Input: [9, 9, 8, 8, 7, 7]
    - Expected Output: -1
    - Justification: There is no number in the array that appears only once.

Solution

To solve this problem, we utilize a hashmap to track the frequency of each number in the given array. The key idea is to iterate through the array, recording the count of each number in the hashmap. Once all elements are accounted for, we scan through the hashmap, focusing on elements with a frequency of one. Among these, we identify the maximum value. This approach ensures that we effectively identify the largest number that appears exactly once in the array, leveraging the hashmap for efficient frequency tracking and retrieval.

1. **Initialization**: Start by creating a hashmap that will be used to store the frequency of each number in the array. This hashmap will be instrumental in identifying numbers that appear only once.
2. **Frequency Count**: Traverse the entire array from the beginning to the end. For each number encountered, increment its count in the hashmap. This step ensures that by the end of the traversal, we have a complete record of how many times each number appears in the array.
3. **Identify Largest Unique Number**: After populating the hashmap, traverse it to identify numbers with a frequency of 1. While doing so, keep track of the largest such number. If no number with a frequency of 1 is found, the result will be -1.
4. **Return Result**: The final step is to return the largest number that has a frequency of 1. If no such number exists, return -1.

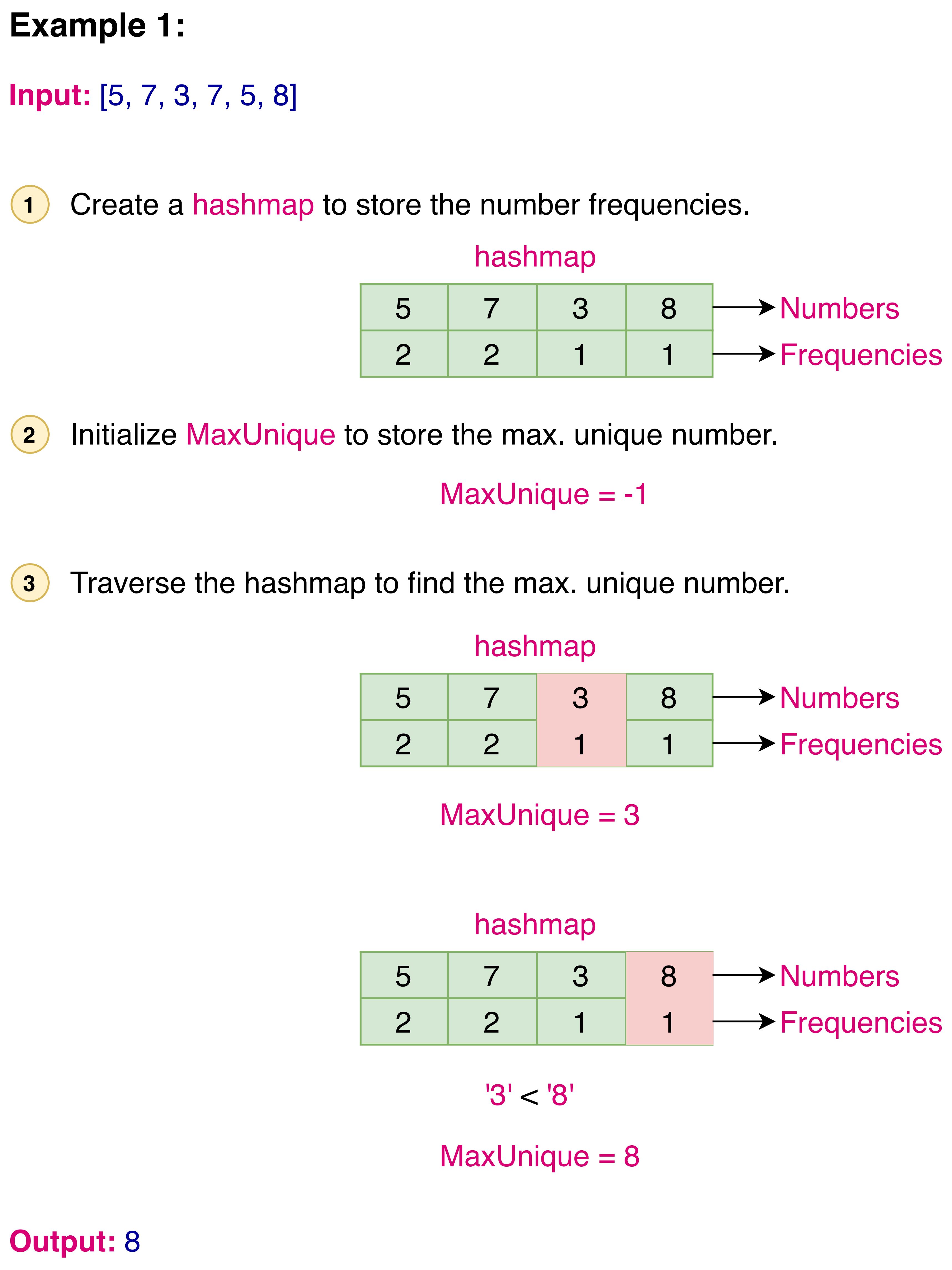
This approach, which leverages the properties of a hashmap, ensures that we can quickly determine the frequency of each number without the need for nested loops or repeated scans of the array.

Algorithm Walkthrough:

Given the input array [5, 7, 3, 7, 5, 8]:

- Initialize a hashmap to store number frequencies.
- Traverse the array:
    - 5 -> frequency is 1
    - 7 -> frequency is 1
    - 3 -> frequency is 1
    - 7 -> frequency is 2
    - 5 -> frequency is 2
    - 8 -> frequency is 1
- Traverse the hashmap:
    - 5 has frequency 2
    - 7 has frequency 2
    - 3 has frequency 1
    - 8 has frequency 1, and it's the largest unique number.

Here is the visual representation of the algorithm:



Example 1:

Input: [5, 7, 3, 7, 5, 8]

① Create a hashmap to store the number frequencies.

5	7	3	8
2	2	1	1

② Initialize MaxUnique to store the max. unique number.

MaxUnique = -1

③ Traverse the hashmap to find the max. unique number.

5	7	3	8
2	2	1	1

MaxUnique = 3

5	7	3	8
2	2	1	1

'3' < '8'

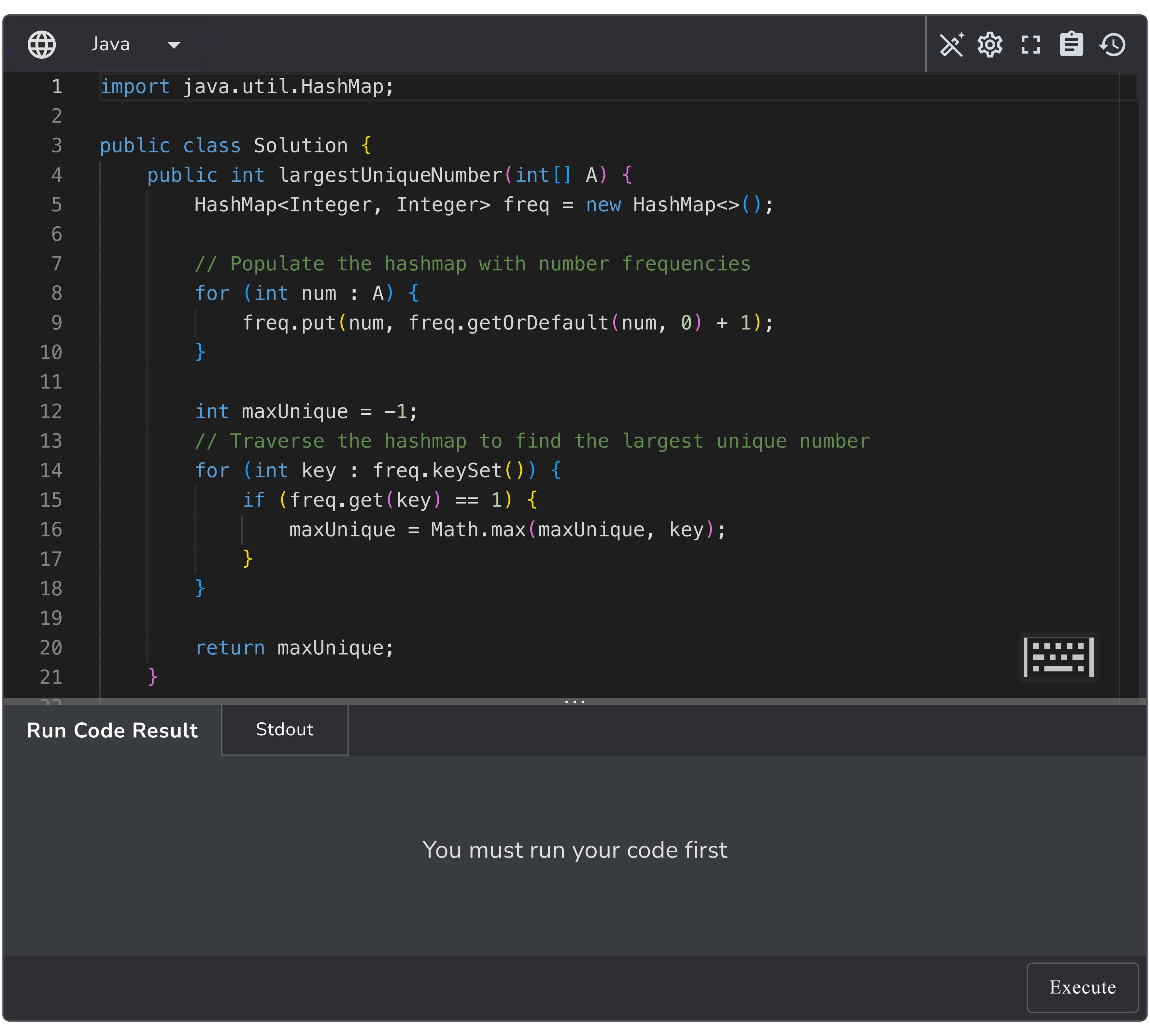
MaxUnique = 8

Output: 8

Largest Unique Number

Code

Here is the code for this algorithm:



```java
import java.util.HashMap;

public class Solution {
    public int largestUniqueNumber(int[] A) {
        HashMap<Integer, Integer> freq = new HashMap<>();

        // Populate the hashmap with number frequencies
        for (int num : A) {
            freq.put(num, freq.getOrDefault(num, 0) + 1);
        }

        int maxUnique = -1;
        // Traverse the hashmap to find the largest unique number
        for (int key : freq.keySet()) {
            if (freq.get(key) == 1) {
                maxUnique = Math.max(maxUnique, key);
            }
        }

        return maxUnique;
    }
```

You must run your code first

Complexity Analysis

Time Complexity: The algorithm traverses the array once to populate the hashmap and then traverses the hashmap to find the largest unique number. Both operations are O(n), where n is the length of the array. Therefore, the overall time complexity is O(n).

Space Complexity: The space complexity is determined by the hashmap, which in the worst case will have an entry for each unique number in the array. Therefore, the space complexity is O(n), where n is the number of unique numbers in the array.

← Previous
Problem 2: Largest Unique Number (Easy)

Next →
Problem 3: Maximum Number Of Balloons (Easy)

☐ Mark as Completed